MAML-Select: An Online Adaptive Client Selection Method for Federated Learning via Meta-Learning

***Abstract*—Federated Learning (FL) enables collaborative model training across distributed edge data, but client selection remains difficult when devices differ in data distribution, compute capacity, energy state, and communication latency. Selecting only the fastest or most statistically useful clients can slow convergence, increase resource cost, or reduce participation coverage. System-aware selectors such as FedCS and TiFL address stragglers, while utility-aware methods such as Oort, FedCor, CriticalFL, and FedGCS use loss, correlation, critical-period, or generative signals to improve training. However, many existing methods rely on fixed scoring rules, costly modelling stages, or feedback that does not explicitly adapt the selector before every round. The remaining gap is a lightweight policy that can rapidly re-rank clients as their utility and system state change during FL. We propose MAML-Select, an online adaptive selector that treats each FL round as a meta-learning task. A compact 6-64-64-1 MLP is adapted with a first-order MAML-style support update from recent local-loss and latency feedback, then used to score available clients before a query-set meta-update. Across Fashion-MNIST, CIFAR-10, and CIFAR-100 with 100 clients, $K = 10$, Dirichlet $\alpha = 0.5$, and three seeds, MAML-Select reduces cumulative TFLOPs by 12.8%, 21.5%, and 1.7% relative to FedAvg, with corresponding energy/carbon reductions of 16.4%, 24.7%, and 4.7%. Accuracy remains competitive with FedAvg and FedGCS, with the largest trade-off on CIFAR-10. These findings suggest that meta-adaptive client selection can make edge AI deployments more resource-conscious while retaining broad client coverage.**

***Impact Statement*—Federated learning is central to privacy-preserving AI in mobile health, intelligent transportation, industrial sensing, and smart-city systems, but these deployments often face tight compute, energy, and latency budgets. MAML-Select contributes an adaptive selection mechanism that reduces unnecessary client computation while preserving broad participation coverage. By using recent training feedback to re-rank clients, the method supports FL systems that respond to changing device conditions instead of relying on a fixed rule. The reported energy and carbon values follow the declared simulator, device-tier, and grid-intensity setup, giving a transparent basis for comparing selection policies. The broader impact is a more resource-aware path for deploying distributed AI at the edge, especially when model updates must remain efficient, reproducible, and privacy-preserving.**

***Index Terms*—Federated learning, client selection, meta-learning, and computational efficiency.**

I. Introduction

FEDERATED Learning (FL) has emerged as an important framework for privacy-preserving distributed intelligence [1]. Clients train models locally and share only model updates. However, they differ in hardware, network conditions, and local data. This creates statistical drift under non-IID data and straggler delays under synchronous training [2]–[6]. Despite these advances, the problem of *whom to select* each round remains open.

Client selection chooses a subset $\mathcal{S}_t \subset \{1, \ldots, N\}$ each round. Client utility is non-stationary: a useful client can become redundant, while an overlooked client can later become important. System-aware methods improve feasible participation but may bias selection away from slow yet data-rich clients [7], [8]. Utility-aware methods prioritize information gain, but may incur high latency or expensive server-side modelling [9]–[13]. Learning-based selectors use histories, features, or reinforcement signals [14]–[16]. An efficient round-by-round mechanism for adapting the ranking policy to evolving utility remains underexplored.

We propose MAML-Select, a client-selection framework based on a MAML-style fast adaptation step [17]. It learns an initialization for a policy network and adapts the policy from recent feedback before the next ranking decision. Unlike RL-based or contextual-bandit selectors, MAML-Select uses rapid gradient-based policy adaptation. Unlike FL personalization [18], it adapts the selector, not the client model. This is useful when devices have limited compute, strict deadlines, or intermittent connectivity. The major contributions can be summarized as follows.

1) We formulate the FL client selection problem as a meta-learning task and introduce the **MAML-Select** algorithm with rapid gradient-based adaptation for online client selection.
2) We demonstrate that MAML-Select *lowers cumulative training TFLOPs* by up to 21.5% and modelled energy/carbon by up to 24.7% relative to FedAvg, with reductions observed on all three datasets.
3) Experiments on Fashion-MNIST, CIFAR-10, and CIFAR-100 under non-IID settings ($\alpha = 0.5$, three seeds) show that MAML-Select delivers accuracy competitive with FedAvg and FedGCS at lower computational cost, and higher accuracy than FedCS, Oort, TiFL, and FedCor in the reported runs; λ-sensitivity and feature-ablation studies support the design choices.

Organization: The rest of this paper is organized as follows. Section II reviews related work on client selection in FL. Section III details the system model and problem formulation. The proposed MAML-Select algorithm is described in Section IV, followed by evaluation and results in Section V. Section VI concludes this paper and discusses future research directions.

II. Related Work

1) System-Aware Selection: System-aware selection addresses stragglers. FedCS selects feasible clients under resource constraints, while TiFL groups devices into online performance tiers [7], [8]. Recent studies show similar concerns in resource-constrained medical IoT and asynchronous FL with

heterogeneous objectives [4], [5]. MAML-Select uses such system signals while learning how much they should matter in the current training phase.

2) Data-Aware and Utility-Based Selection: Data-aware methods select clients for model utility rather than only speed. Oort prioritizes useful and fast clients, and FedCor models correlations between client losses [9], [10]. CriticalFL uses critical learning periods, while FedGCS searches for client combinations in a learned continuous space [12], [13]. Other work couples selection with compression or aggregation design [11], [19]. MAML-Select instead targets rapid adaptation of the ranking policy as client utility evolves.

3) Learning-Based and Meta-Learning Approaches: Learning-based selectors improve from observed feedback. FAVOR uses deep Q-learning, while neural contextual bandits learn from client features and rewards [14], [15]. Recent work also studies deep-RL selection for robustness and fairness-aware inclusive participation [16], [20]. Personalized FL is related but adapts models to clients; MAML-Select adapts the server-side ranking policy [18].

Table I consolidates these distinctions. For methods without a stated closed-form selection complexity, it reports the dominant server-side operation.

Difference from prior work: MAML-Select meta-learns the selection policy h_ϕ. Unlike heuristic, RL, contextual-bandit, and generative selectors, it uses a MAML inner update to adapt the ranking policy from recent feedback before the next decision. Its contribution is rapid policy adaptation under evolving client utility, not personalized model training.

III. System Model and Problem Formulation

We consider a central server and N heterogeneous clients indexed by $\mathcal{N} = \{1, 2, \ldots, N\}$. Client i stores a local dataset $\mathcal{D}_i$ with n_i samples. The global objective is to minimize the empirical risk over the distributed data as

$$\min_{\theta \in \mathbb{R}^d} F(\theta) = \sum_{i=1}^{N} \frac{n_i}{n} F_i(\theta), \tag{1}$$

where $\theta \in \mathbb{R}^d$ denotes the global model, $n = \sum_{i=1}^{N} n_i$, and $F_i(\theta) = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell(\theta; x_{i,j}, y_{i,j})$ is the local empirical risk.

At round t, let $\mathcal{A}_t \subseteq \mathcal{N}$ denote the available clients and $\mathcal{S}_t \subseteq \mathcal{A}_t$ the selected cohort. The server maintains the global model and the selection policy, while clients keep their data locally and return only model updates, local loss summaries, and resource-state feedback. Selection is therefore a server-side decision made before each broadcast, under the usual cross-device FL constraint that only a small fraction of available clients can train in a round.

Client i has latency $T_{i,t} = T_{i,t}^{comp} + T_{i,t}^{comm}$, where $T_{i,t}^{comp} = \frac{En_iC_{model}}{f_{i,t}}$ and $T_{i,t}^{comm} = \frac{M}{B_{i,t}} + \delta_{i,t}$. Here, E is the number of local epochs, C_{model} is the number of FLOPs per sample, $f_{i,t}$ is the compute rate, M is the update size, $B_{i,t}$ is the uplink bandwidth, and $\delta_{i,t}$ is the propagation delay. In synchronous FL, the slowest selected client determines the round latency:

$$T_t^{round} = \max_{i \in \mathcal{S}_t} T_{i,t}, \tag{2}$$

We report simulated latency and TFLOPs as computation indicators and derive energy/carbon estimates from the stated device-tier assumptions.

Let $a_{i,t}$ indicate whether client i is selected. Under policy π_ϕ, the constrained client-selection problem is

$$\begin{aligned} \min_{\pi_\phi} \quad & \sum_{t=1}^{T} \left[F(\theta_{t+1}) + \lambda T_t^{round} \right] \\ \text{s.t.} \quad & \sum_{i=1}^{N} a_{i,t} = K, \quad a_{i,t} \in \{0,1\}, \\ & a_{i,t} \le \mathbf{1}\{i \in \mathcal{A}_t\}, \quad \mathcal{S}_t = \{i : a_{i,t} = 1\}. \end{aligned} \tag{3}$$

Here, K is the cohort size, T is the number of rounds, and $\lambda \ge 0$ controls the latency–loss trade-off. A larger λ places more emphasis on latency, while $\lambda = 0$ yields loss-only selection. We assume $|\mathcal{A}_t| \ge K$. The model θ_{t+1} results from local training and aggregation over $\mathcal{S}_t$. The problem remains challenging because the action space is combinatorial and client utility evolves during training.

IV. Methodology: MAML-Select Algorithm

Building on (3), MAML-Select treats client selection as a meta-learning problem. Client utility changes as training progresses. However, the mechanism for selecting useful clients from recent feedback can be learned.

1) State Space: The server constructs a state vector $\boldsymbol{s}_{i,t} \in \mathbb{R}^6$ for each client i at round t. It contains local training loss $\hat{L}_{i,t}$, gradient norm $\|\nabla F_i\|_2$, estimated latency $\hat{T}_{i,t}$, battery level $b_{i,t}$, selection frequency $q_{i,t}$, and staleness $\tau_{i,t}$ (rounds since last selected) [11], [15]. Loss and gradient norm characterize statistical utility. Latency and battery level characterize system constraints. Selection frequency and staleness expose participation imbalance. Each FL round t is treated as a task $\mathcal{T}_t$. The goal is to rank clients so that the top-K cohort minimizes the per-round cost.

MAML-Select trains a meta-policy network $h_\phi(\boldsymbol{s}_{i,t})$, parameterized by ϕ, that learns an initialization adaptable to the current round using data from the preceding round. The policy is a 6-64-64-1 multilayer perceptron with ReLU activations and 4,673 trainable parameters. The support set represents the past experience: $\mathcal{D}_{sup}(t) = \{(\boldsymbol{s}_{i,t-1}, c_{i,t-1}) \mid i \in \mathcal{S}_{t-1}\}$, containing client states and observed costs from round $t-1$. The candidate set represents the current decision: $\mathcal{D}_{cand}(t) = \{\boldsymbol{s}_{i,t} \mid i \in \mathcal{A}_t\}$, containing the states of available clients.

2) Cost Function: The per-client cost $c_{i,t}$ supplies feedback to the ranking policy. Consistent with (3), it is defined as

$$c_{i,t} = \lambda \underbrace{\max(0, T_{i,t} - T_{target})}_{\text{latency penalty}} - \underbrace{\Delta_{i,t}}_{\text{local loss reduction}}. \tag{4}$$

Here, $\Delta_{i,t} = \hat{L}_{i,t}^{before} - \hat{L}_{i,t}^{after}$ is the observed local loss reduction for selected client i. It is a client-level proxy for statistical utility and does not require a server-held validation set. The target T_{target} is a soft deadline. Minimizing $c_{i,t}$ favors lower latency and larger local loss reduction.

A. The MAML-Select Algorithm

The algorithm proceeds in three phases per round:

TABLE I
COMPARISON OF REPRESENTATIVE METHODS AND MAML-SELECT.

Method	Primary objective	Adaptation mechanism	Selection complexity or dominant operation	Optimization strategy
FedCS [7]	Increase feasible updates under resource constraints	Resource estimates before selection	Resource-constrained subset search	Deadline-aware scheduling
TiFL [8]	Mitigate stragglers while preserving accuracy	Online tier updates	Tiering and probabilistic sampling	Adaptive tier-based selection
Oort [9]	Improve time-to-accuracy	Utility and speed feedback	Utility scoring and guided sampling	Exploration and exploitation
FedCor [10]	Accelerate convergence under non-IID data	Loss-correlation model	Gaussian-Process covariance operations	GP-based active selection
FAVOR [14]	Counterbalance non-IID bias	Experience-driven policy updates	DQN inference and training	Deep Q-learning
NCCB [15]	Learn contextual client combinations	Client features and reward feedback	LSH feature extraction and bandit updates	Neural contextual combinatorial bandit
FedCG [11]	Couple selection and gradient compression	Capability-aware compression updates	Submodular optimization and linear programming	Joint iterative optimization
CriticalFL [12]	Exploit critical learning periods	Period-aware participation	Base-selector dependent	Selector augmentation
FedGCS [13]	Optimize client combinations	Search in a learned continuous space	Latent-space optimization and beam search	Gradient-based generative selection
Kazemi et al. [16]	Improve robustness under poisoning attacks	Experience-driven policy updates	Deep-RL inference and training	Deep reinforcement learning
Vox Populi [20]	Promote fair and inclusive participation	Fairness-aware client selection	Framework-dependent	Inclusive selection framework
MAML-Select	Adapt rankings under evolving client utility	Fast adaptation from recent round feedback	$\mathcal{O}(N\|\phi\|)$ scoring plus a gradient-adaptation step	Gradient-based meta-learning

Phase 1 (Inner Loop Adaptation (Look-Ahead)): At the start of round t, the server adapts the meta-policy ϕ_t on $\mathcal{D}_{sup}(t)$ using one gradient-descent step:

$$\phi_t' = \phi_t - \beta \nabla_{\phi_t} \mathcal{L}_{sup}(h_{\phi_t}; \mathcal{D}_{sup}(t)), \tag{5}$$

where $\mathcal{L}_{sup}$ is the MSE loss between predicted scores and observed costs from round $t-1$, and β is the inner learning rate.

Statement 1. *Let* $g_t(\phi) = \mathcal{L}_{sup}(h_\phi; \mathcal{D}_{sup}(t))$. *If* g_t *is* L*-smooth and* $0 < \beta < 2/L$, *the inner update in* (5) *satisfies*

$$g_t(\phi_t') \le g_t(\phi_t) - \beta \left(1 - \frac{L\beta}{2}\right) \|\nabla g_t(\phi_t)\|_2^2.$$

This standard descent bound shows why recent feedback can improve the round-specific policy before ranking current candidates. It motivates rapid adaptation. It is not a global FL convergence guarantee.

Phase 2 (Client Selection): The adapted policy $h_{\phi_t'}$ scores the available clients and selects the K clients with the lowest predicted costs:

$$\mathcal{S}_t = \text{Top-}K_{i \in \mathcal{A}_t} \left(-h_{\phi_t'}(\boldsymbol{s}_{i,t})\right). \tag{6}$$

Phase 3 (FL Execution and Outer Loop Update): The selected clients perform standard FL training. Their observed costs form $\mathcal{D}_{query}(t) = \{(\boldsymbol{s}_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$. The query loss is also mean squared error. The meta-policy is updated on this query set:

$$\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi_t'} \mathcal{L}_{query}(h_{\phi_t'}; \mathcal{D}_{query}(t)), \tag{7}$$

where η is the meta-learning rate. We use a first-order MAML-style approximation. It applies the query gradient at ϕ_t' and omits second-order derivatives. The full procedure is detailed in Algorithm 1 and Fig. 1.

MAML-Select differs from fixed-rule selectors because its ranking policy is adapted before each decision. It also differs from RL selectors because the update is an explicit gradient step on recent support feedback. The method is designed for rapidly changing client utility.

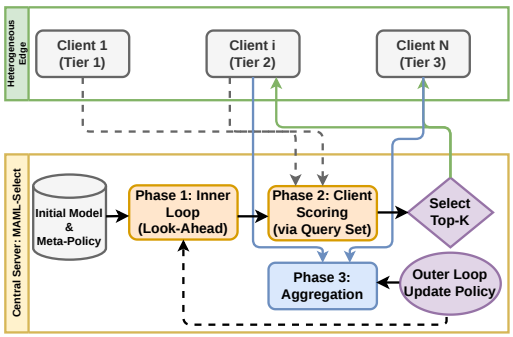

Fig. 1. System model and MAML-Select workflow. A central server observes client states, selects K clients from the available pool, aggregates their local updates, and uses returned local-loss and system feedback for support-set adaptation, candidate scoring, and query-set meta-update.

Algorithm 1 MAML-Select Algorithm

Initialize: Global model θ_0; meta-policy parameters ϕ; inner LR β; outer LR η; trade-off λ.
for round $t = 1, \ldots, T$ **do**
  Server broadcasts global model θ_t to all clients.
  Collect state vectors $\boldsymbol{s}_{i,t}$ from available clients.
  if $t > 1$ **then**
    `// Phase 1: Inner Loop Adaptation`
    Construct $\mathcal{D}_{sup}$ from round $t-1$ results.
    Compute fast weights ϕ_t' using Eq. 5.
  else
    $\phi_t' \leftarrow \phi_t$ `// No adaptation for the first round`
  end if
  `// Phase 2: Client Selection`
  Score clients: $\text{score}_i = h_{\phi_t'}(\boldsymbol{s}_{i,t})$ for all $i \in \mathcal{A}_t$.
  Select lowest-cost clients: $\mathcal{S}_t \leftarrow$ Top-$K(-\text{scores})$.
  `// FL Training Round`
  Receive local updates $\{\Delta\theta_{i,t}\}_{i \in \mathcal{S}_t}$, local losses, and system metrics.
  Aggregate: $\theta_{t+1} \leftarrow \theta_t + \sum_{i \in \mathcal{S}_t} \frac{n_i}{\sum_{j \in \mathcal{S}_t} n_j} \Delta\theta_{i,t}$.
  `// Phase 3: Outer Loop Meta-Update`
  Compute costs $c_{i,t}$ using Eq. 4 for $i \in \mathcal{S}_t$.
  Construct $\mathcal{D}_{query}(t) = \{(\boldsymbol{s}_{i,t}, c_{i,t}) \mid i \in \mathcal{S}_t\}$.
  Meta-update: $\phi_{t+1} \leftarrow \phi_t - \eta \nabla_{\phi_t'} \mathcal{L}_{query}(h_{\phi_t'}; \mathcal{D}_{query}(t))$.
end for

1) Computational Complexity: Let $P = |\phi|$ denote the number of meta-policy parameters. For the 6-64-64-1 MLP used here, $P = 4{,}673$. A forward pass for one client costs $\mathcal{O}(P)$, so scoring N available clients costs $\mathcal{O}(NP)$. Selecting the best K clients can be implemented with a size-K heap in

$\mathcal{O}(N \log K)$, or by partial sorting. The inner support update and outer query update each use at most the selected cohort and therefore cost $\mathcal{O}(KP)$. Thus the per-round server-side selector cost is

$$\mathcal{O}(NP + N \log K + 2KP)\,. \tag{8}$$

Since P and K are fixed in the experimental protocol ($P = 4{,}673$, $K = 10$), MAML-Select scales linearly with the client pool size N. The memory cost is $\mathcal{O}(P+N+K)$ for the policy parameters, client scores, and selected cohort. This contrasts with correlation-matrix or Gaussian-Process selection stages whose covariance operations can grow superlinearly, and in common FedCor-style implementations cubically, with N.

V. Evaluation and Results

We evaluate on three benchmarks: Fashion-MNIST using LightCNN-9 [21] (lightweight IoT task), CIFAR-10 using ResNet18 [22] (complex vision task), and CIFAR-100 using ResNet18 (100-class fine-grained task). $N = 100$ clients are simulated with non-IID data partition (Dirichlet $\alpha = 0.5$), selecting $K = 10$ clients per round for $T = 200$ rounds (150 rounds for CIFAR-100) with $E = 5$ local epochs and batch size 32. All results are reported as the mean over three random seeds $(42, 123, 2026)$. The standard deviations are shown for the three seeds. Local models use PyTorch default initialization and SGD with learning rate 0.01, momentum 0.9, weight decay 10^{-4}, and a constant scheduler. The meta-policy uses one inner step with $\beta = 0.01$ and Adam for the outer update with $\eta = 0.001$; the selector seed is fixed to 2026. Compute rates follow three device tiers inspired by mobile benchmarking studies [23]: Tier 1 (20%, $1.0\times$), Tier 2 (50%, $2.0\times$), and Tier 3 (30%, $4.0\times$). The target latency deadline T_{target} is set to the average round latency of Tier 2 clients, and the trade-off parameter $\lambda = 0.5$. Baselines include FedAvg (random selection), FedCS (system-aware deadline-constrained), Oort (utility-based with importance sampling), TiFL (tier-based), FedCor (correlation-based via Gaussian Processes), FedGCS (generative selection), and CriticalFL (critical-period cohort scaling).

A. Accuracy and Resource Cost

We estimate the client-training computation per round as

$$\text{TFLOPs}_{total} = \sum_{t=1}^{T} \sum_{i \in \mathcal{S}_t} E \cdot n_i \cdot C_{model} \times 10^{-12}, \tag{9}$$

where C_{model} is the per-sample cost; for ResNet18 on CIFAR-10 a client round is approximately 4.13 TFLOPs. Modelled energy and carbon scale this cost by per-tier device power and a declared grid intensity; carbon-accounting metadata and emission units follow CodeCarbon v2.4.1 [24].

Table II summarises all methods. MAML-Select attains accuracy on par with FedAvg and FedGCS on Fashion-MNIST (90.1% vs. 90.2% and 90.7%) and CIFAR-100 (58.2% vs. 59.7% and 58.7%), and has higher accuracy than FedCS, Oort, TiFL, and FedCor in the reported runs. On CIFAR-10 it trades an accuracy reduction (75.6% vs. FedAvg 79.4%) for substantially lower cost. On CIFAR-100, CriticalFL reaches

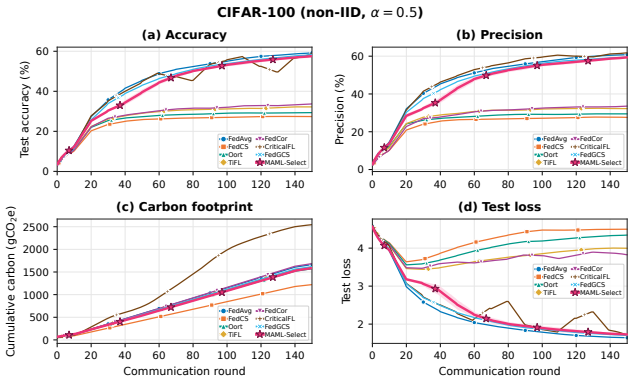

Fig. 2. CIFAR-100 convergence over 150 rounds. The panels report accuracy, precision, cumulative modelled carbon, and test loss.

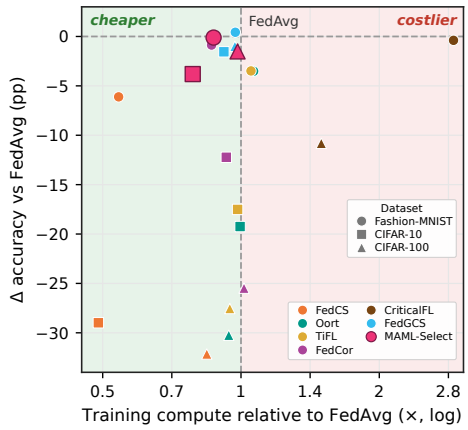

Fig. 3. Efficiency–accuracy trade-off relative to FedAvg. $x < 1$ indicates lower cumulative training compute, while y reports accuracy change in percentage points.

48.8% accuracy with the highest TFLOPs and modelled resource cost among the completed methods, whereas MAML-Select is more accurate and substantially cheaper. The meta-update overhead ($\varepsilon \approx 0.001$ GFLOPs per round) is negligible relative to client training, and all selectors transmit identical communication volume.

Fig. 2 summarises the CIFAR-100 round-wise behaviour. MAML-Select remains close to FedAvg and FedGCS on accuracy, precision, and loss while staying in the efficient carbon cluster. CriticalFL is costlier without an accuracy advantage in this setting.

Fig. 3 shows the efficiency–accuracy trade-off relative to FedAvg. Points to the left of $x = 1$ use less cumulative training compute; points above $y = 0$ improve accuracy. MAML-Select is consistently in the lower-compute region. Relative to FedAvg over shared seeds, it lowers cumulative TFLOPs by $12.8\%/21.5\%/1.7\%$ and modelled energy and carbon by $16.4\%/24.7\%/4.7\%$ on Fashion-MNIST/CIFAR-10/CIFAR-100. The corresponding accuracy changes are $-0.1/-3.8/-1.5$ percentage points. Thus the main empirical gain is resource efficiency with competitive, not universally superior, accuracy. This is the desired operating region for resource-constrained FL: the selector avoids the high-cost cohorts used by CriticalFL while retaining the data coverage needed for stable learning.

TABLE II
BENCHMARK SUMMARY ACROSS NON-IID DATASETS. ACCURACY, PRECISION, RECALL, F1, AND COVERAGE ARE IN PERCENT; ENERGY IS IN WATT-HOUR (WH) AND CARBON IS IN GRAMS OF CARBON DIOXIDE EQUIVALENT (gCO_2e).

Method	Acc.	Prec.	Rec.	F1	TFLOPs	Energy	Carbon	Jain	Cov.
Fashion-MNIST									
FedAvg	90.22±0.58	90.55±0.20	90.22±0.58	90.22±0.51	140±1	5752±72	2732±34	0.95±0.00	100±0
FedCS	84.11±1.82	84.84±0.77	84.11±1.82	83.91±1.98	76±7	2772±211	1317±100	0.10±0.00	10±0
Oort	86.70±0.19	86.66±0.21	86.70±0.19	86.42±0.26	149±8	5963±456	2832±217	0.10±0.00	10±0
TiFL	86.73±0.34	86.62±0.26	86.73±0.34	86.51±0.34	147±6	6062±348	2879±165	0.11±0.00	12±0
FedCor	89.35±0.50	89.31±0.51	89.35±0.50	89.22±0.57	121±5	4845±204	2301±97	0.26±0.04	49±8
CriticalFL	89.83±0.00	89.98±0.00	89.83±0.00	89.81±0.00	405±0	16696±0	7931±0	0.99±0.00	100±0
FedGCS	90.65±0.29	90.73±0.16	90.65±0.29	90.63±0.26	136±4	5595±113	2658±54	0.95±0.02	100±0
MAML-Select	90.11±0.47	90.43±0.04	90.11±0.47	90.04±0.30	122±2	4809±79	2284±37	0.77±0.06	100±0
CIFAR-10									
FedAvg	79.43±1.84	80.01±1.28	79.43±1.84	79.36±1.78	8341±52	4798±24	2279±11	0.95±0.00	100±0
FedCS	50.45±2.57	50.88±2.60	50.45±2.57	49.87±2.33	4081±600	2073±262	985±125	0.10±0.00	10±0
Oort	60.19±5.18	60.84±4.97	60.19±5.18	59.69±4.61	8303±1923	4677±856	2222±406	0.10±0.00	10±0
TiFL	61.93±5.04	62.48±4.78	61.93±5.04	61.36±4.60	8200±2143	4800±1203	2280±572	0.11±0.00	12±0
FedCor	67.20±5.33	67.83±4.74	67.20±5.33	66.81±5.37	7748±695	4452±435	2115±206	0.20±0.02	31±5
CriticalFL									
FedGCS	77.88±1.75	78.93±1.50	77.88±1.75	77.69±1.54	7656±278	4428±154	2103±73	0.90±0.03	100±0
MAML-Select	75.63±1.43	76.51±1.55	75.63±1.43	75.25±1.34	6549±118	3610±60	1715±29	0.61±0.01	100±0
CIFAR-100									
FedAvg	59.66±0.29	60.90±0.48	59.66±0.29	59.13±0.40	6331±6	3626±18	1722±9	0.94±0.00	100±0
FedCS	27.52±0.29	27.93±0.10	27.52±0.29	26.39±0.09	5334±23	2659±12	1263±6	0.10±0.00	10±0
Oort	29.41±1.73	29.82±1.77	29.41±1.73	28.13±1.73	5955±189	3421±181	1625±86	0.10±0.00	10±0
TiFL	32.12±0.30	33.07±0.50	32.12±0.33	31.02±0.42	5986±28	3499±14	1662±5	0.11±0.00	12±0
FedCor	34.18±0.24	33.89±0.26	34.18±0.60	32.93±0.20	6428±32	3668±18	1742±3	0.12±0.00	14±0
CriticalFL	48.84±1.58	60.56±1.89	48.84±1.58	48.07±2.32	9469±392	5438±214	2583±102	0.97±0.00	100±0
FedGCS	58.66±0.11	60.24±0.03	58.66±0.11	58.08±0.20	6140±24	3526±4	1675±2	0.82±0.02	100±0
MAML-Select	58.15±0.51	59.87±0.55	58.15±0.51	57.66±0.33	6224±20	3457±6	1642±3	0.78±0.03	100±0

MAML-Select reaches 100% client coverage on every dataset. Its Jain index (0.61–0.78) is below FedAvg and FedGCS, so we do not claim improved fairness; the coverage result instead shows that the policy does not permanently exclude clients.

B. Selector Scaling

To verify (8), we measured MAML-Select selection overhead for 50 rounds while increasing $N = \{20, 40, 80, 100\}$ and keeping $K/N = 0.1$. Fig. 4 focuses on the proposed selector: the measured server-side overhead remains about 11–13 ms per round, while the normalized analytical work follows the expression $NP+N\log K+2KP$. Under the main experimental setting with fixed $K = 10$ and fixed policy size $P = 4{,}673$, the dominant term is linear in N. This cost is small relative to local client training.

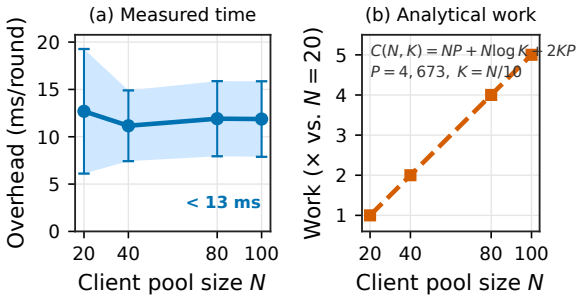

Fig. 4. MAML-Select selector scaling. Panel (a) reports measured overhead with 95% confidence intervals over 10 rounds. Panel (b) reports the normalized operation count from Eq. 8.

C. Sensitivity and Ablation

Fig. 5 analyses λ on Fashion-MNIST using four normalized curves: accuracy, compute, energy, and fairness. Final accuracy remains stable across $\lambda \in \{0.1, 0.5, 1, 5\}$, with about a 1.0 percentage-point spread, while compute and energy decrease as the latency/resource term receives more weight. Fairness decreases more sharply for very large λ, indicating a stronger efficiency–fairness trade-off. We therefore use $\lambda = 0.5$ as a balanced operating point: it keeps accuracy close to the best observed value while reducing computation relative to $\lambda = 0.1$.

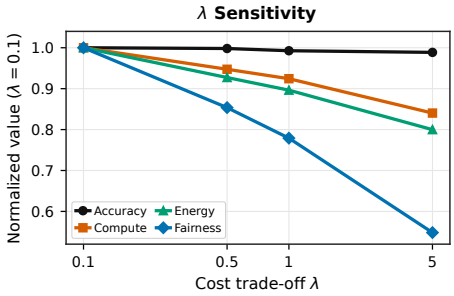

Fig. 5. λ sensitivity on Fashion-MNIST. Curves are normalized to the value at $\lambda = 0.1$ and report accuracy, compute, energy, and fairness.

Fig. 6 and Table III show the state-feature ablation. Removing a single feature changes Fashion-MNIST accuracy by less than about 0.35 percentage points. The largest negative shifts occur when battery and loss information are removed, showing that the policy benefits from both system and statistical feedback. Removing frequency or staleness can slightly increase accuracy but lowers fairness/coverage behaviour, so these features remain important for stable participation.

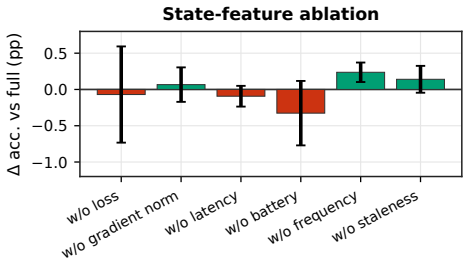

Fig. 6. State-feature ablation on Fashion-MNIST. Each bar reports the accuracy change relative to the full six-feature state vector.

TABLE III
STATE-FEATURE ABLATION ON FASHION-MNIST ($n = 3$). EACH ROW REMOVES ONE FEATURE FROM THE SIX-DIMENSIONAL STATE VECTOR; THE FULL VECTOR IS THE REFERENCE.

Variant	Acc. (%)	TFLOPs	Jain
Full state vector	90.23 ± 0.55	122	0.78
w/o loss	90.16 ± 0.66	123	0.79
w/o gradient norm	90.30 ± 0.24	122	0.78
w/o latency	90.14 ± 0.14	119	0.70
w/o battery	89.90 ± 0.44	122	0.79
w/o frequency	90.47 ± 0.13	115	0.65
w/o staleness	90.37 ± 0.19	123	0.78

VI. CONCLUSION AND FUTURE WORK

We introduced MAML-Select, an online adaptive client-selection framework for heterogeneous FL. It learns a lightweight ranking policy and adapts it from recent feedback before each selection decision. Across Fashion-MNIST, CIFAR-10, and CIFAR-100, MAML-Select delivers competitive accuracy while lowering cumulative computation, modelled energy, and carbon. The λ-sensitivity and feature-ablation studies support the design choices; future work will add analysis on sound datasets with large-scale benchmarks.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.

[2] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5132–5143.

[3] V. S. Mai, R. J. La, and T. Zhang, "A study of enhancing federated learning on non-IID data with server learning," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 11, pp. 5589–5604, 2024.

[4] S. Tapwal, "Stragglers reimagined: Explainability-driven adaptive federated learning for resource constrained IoMT system," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1002–1011, 2026.

[5] A. Forootani, R. Iervolino, and M. Tipaldi, "Asynchronous federated learning with nonconvex client objective functions and heterogeneous dataset," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 2610–2624, 2026.

[6] P. Borazjani, M. S. Fouladgar, and R. Ghanbarzadeh, "Redefining non-IID data in federated learning for computer vision tasks," *IEEE Transactions on Artificial Intelligence*, 2026, early Access.

[7] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *IEEE International Conference on Communications (ICC)*. IEEE, 2019, pp. 1–7.

[8] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, and Y. Cheng, "TiFL: A tier-based federated learning system," in *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*, 2020, pp. 125–136.

[9] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *15th USENIX Symposium on Operating Systems Design and Implementation*, 2021, pp. 19–35.

[10] M. Tang, X. Ning, Y. Wang, J. Sun, Y. Wang, H. Li, and Y. Chen, "FedCor: Correlation-based active client selection strategy for heterogeneous federated learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2022, pp. 10 102–10 111.

[11] Y. Xu, Z. Jiang, H. Xu, Z. Wang, C. Qian, and C. Qiao, "Federated learning with client selection and gradient compression in heterogeneous edge systems," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5446–5461, 2024.

[12] G. Yan, H. Wang, X. Yuan, and J. Li, "CriticalFL: A critical learning periods augmented client selection framework for efficient federated learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 5034–5044.

[13] Z. Ning, C. Tian, M. Xiao, W. Fan, P. Wang, L. Li, P. Wang, and Y. Zhou, "FedGCS: A generative framework for efficient client selection in federated learning via gradient-based optimization," in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, 2024, pp. 4760–4768.

[14] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-IID data with reinforcement learning," in *IEEE INFOCOM*. IEEE, 2020, pp. 1698–1707.

[15] Q. Pan, H. Cao, Y. Zhu, J. Liu, and B. Li, "Contextual client selection for efficient federated learning over edge devices," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 6538–6548, 2024.

[16] F. Kazemi, A. Badiei, and H. Kebriaei, "Robust peer-to-peer federated learning with deep reinforcement learning based client selection against data poisoning attacks," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 262–271, 2026.

[17] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 1126–1135.

[18] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," *Advances in Neural Information Processing Systems*, vol. 33, pp. 3557–3568, 2020.

[19] N. T. Tran, L. H. T. Tam, and M. T. Thai, "FedImp: Enhancing federated learning convergence with impurity-based weighting aggregation," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1652–1665, 2026.

[20] S. Arisdakessian, O. A. Wahab, A. Mourad, and H. Otrok, "Towards vox populi in federated learning: A fair and inclusive client selection framework," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 1997–2011, 2026.

[21] X. Wu, R. He, Z. Sun, and T. Tan, "A light CNN for deep face representation with noisy labels," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 11, pp. 2884–2896, 2018.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2016, pp. 770–778.

[23] A. Ignatov, R. Timofte, A. Kulik, S. Yang, K. Wang, F. Baum, M. Wu, L. Xu, and L. Van Gool, "AI Benchmark: Running deep neural networks on android smartphones," in *European Conference on Computer Vision Workshops*, 2018, pp. 288–314.

[24] B. Courty, V. Schmidt, S. Luccioni, Goyal-Kamal, MarionCoutarel, B. Feld, J. Lecourt, LiamConnell, A. Saboni, Inimaz, supatomic, M. Leval, L. Blanche, A. Cruveiller, ouminasara, F. Zhao, A. Joshi, A. Bogroff, H. de Lavoreille, N. Laskaris, E. Abati, D. Blank, Z. Wang, A. Catovic, M. Alencon, M. Stechly, C. Bauer, L. O. N. de Araujo, JPW, and MinervaBooks, "mlco2/codecarbon: v2.4.1," May 2024. [Online]. Available: https://doi.org/10.5281/zenodo.11171501